Status	Finished
Started	Thursday, 29 October 2026, 7:56 PM
Completed	Thursday, 29 October 2026, 8:04 PM
Duration	8 mins 1 sec
Grade	**10.00** out of 10.00 (**100**%)

Question **1**

Correct

Mark 1.00 out of 1.00

Write a Python program to sort a list of elements using the merge sort algorithm.

For example:

Input	Result
5 6 5 4 3 8	3 4 5 6 8

Answer: (penalty regime: 0 %)

```
import sys

def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left_half = merge_sort(arr[:mid])
    right_half = merge_sort(arr[mid:])

    return merge(left_half, right_half)

def merge(left, right):
    sorted_arr = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            sorted_arr.append(left[i])
            i += 1
        else:
            sorted_arr.append(right[j])
            j += 1

    sorted_arr.extend(left[i:])
    sorted_arr.extend(right[j:])

    return sorted_arr

def main():
    # Read all tokens from standard input (handles multiple test cases or multi-line inputs)
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    iterator = iter(input_data)

    # Continuously process test cases
    for num_elements_str in iterator:
        n = int(num_elements_str)
        elements = [int(next(iterator)) for _ in range(n)]

        sorted_elements = merge_sort(elements)
        # Print elements separated by spaces to match expected output format exactly
        print(*(sorted_elements))

if __name__ == "__main__":
    main()
```

	Input	Expected	Got	
✓	5 6 5 4 3 8	3 4 5 6 8	3 4 5 6 8	✓
✓	9 14 46 43 27 57 41 45 21 70	14 21 27 41 43 45 46 57 70	14 21 27 41 43 45 46 57 70	✓
✓	4 86 43 23 49	23 43 49 86	23 43 49 86	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **2**

Correct

Mark 1.00 out of 1.00

Given an listof integers, sort the array in ascending order using the *Bubble Sort* algorithm above. Once sorted, print the following three lines:

1. List is sorted in numSwaps swaps., where numSwaps is the number of swaps that took place.
2. First Element: firstElement, the *first* element in the sorted list.
3. Last Element: lastElement, the *last* element in the sorted list.

For example, given a worst-case but small array to sort: a=[6,4,1]. It took 3 swaps to sort the array. Output would be

```
Array is sorted in 3 swaps.
```

```
First Element: 1
```

```
Last Element: 6
```

Input Format

The first line contains an integer,n , the size of the list a .
The second line contains n, space-separated integers a[i].

Constraints

- 2<=n<=600
- 1<=a[i]<=$2x10^6$.

Output Format

You must print the following three lines of output:

1. List is sorted in numSwaps swaps., where numSwaps is the number of swaps that took place.
2. First Element: firstElement, the *first* element in the sorted list.
3. Last Element: lastElement, the *last* element in the sorted list.

Sample Input 0

3

1 2 3

Sample Output 0

List is sorted in 0 swaps.

First Element: 1

Last Element: 3

For example:

Input	Result
3 3 2 1	List is sorted in 3 swaps. First Element: 1 Last Element: 3
5 1 9 2 8 4	List is sorted in 4 swaps. First Element: 1 Last Element: 9

Answer: (penalty regime: 0 %)

```
import sys

def solve():
    # Read all tokens from standard input
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    iterator = iter(input_data)

    # Process test cases until input ends
    while True:
        try:
            n_str = next(iterator)
        except StopIteration:
            break

        n = int(n_str)
        a = [int(next(iterator)) for _ in range(n)]

        num_swaps = 0

        # Bubble Sort algorithm with swap counting
        for i in range(n):
            for j in range(n - 1):
                if a[j] > a[j + 1]:
                    a[j], a[j + 1] = a[j + 1], a[j]
                    num_swaps += 1

        # Print output in the required format
        print(f"List is sorted in {num_swaps} swaps.")
        print(f"First Element: {a[0]}")
        print(f"Last Element: {a[-1]}")

if __name__ == "__main__":
    solve()
```

	Input	Expected	Got	
✓	3 3 2 1	List is sorted in 3 swaps. First Element: 1 Last Element: 3	List is sorted in 3 swaps. First Element: 1 Last Element: 3	✓
✓	5 1 9 2 8 4	List is sorted in 4 swaps. First Element: 1 Last Element: 9	List is sorted in 4 swaps. First Element: 1 Last Element: 9	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **3**

Correct

Mark 1.00 out of 1.00

The problem is that we want to reverse a array in O(N) linear time complexity and we want the algorithm to be in-place as well!
For example: input is [1,2,3,4,5] then the output is [5,4,3,2,1]
Input
5
1 2 3 4 5
Output
5 4 3 2 1

For example:

Input	Result
5 1 2 3 4 5	5 4 3 2 1

Answer: (penalty regime: 0 %)

```
import sys

def reverse_array_in_place(arr):
    left = 0
    right = len(arr) - 1

    # Swap elements from both ends moving toward the center
    while left < right:
        arr[left], arr[right] = arr[right], arr[left]
        left += 1
        right -= 1

def main():
    # Read all tokens from standard input
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    iterator = iter(input_data)

    # Process inputs continuously
    for n_str in iterator:
        n = int(n_str)
        arr = [int(next(iterator)) for _ in range(n)]

        reverse_array_in_place(arr)

        # Print elements space-separated as expected
        print(*arr)

if __name__ == "__main__":
    main()
```

	Input	Expected	Got	
✓	5 1 2 3 4 5	5 4 3 2 1	5 4 3 2 1	✓
✓	10 0 2 4 6 8 1 3 5 7 9	9 7 5 3 1 8 6 4 2 0	9 7 5 3 1 8 6 4 2 0	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **4**

Correct

Mark 1.00 out of 1.00

Objective:

Develop a Python program to find the k-th maximum value in a given list of integers. The program should handle various edge cases, including lists with duplicate values, empty lists, and invalid values of k. The k-th maximum value refers to the k-th largest distinct element in the list.

Background:

Finding the k-th maximum value in a list is a common problem in computer science, often encountered in fields like data analysis, competitive programming, and software development. This problem requires an understanding of sorting algorithms, data structures, and efficient problem-solving techniques. By solving this problem, one gains insights into how to handle large datasets and optimize performance in practical applications.

Problem Description:

Given a list of integers, the task is to determine the k-th maximum value in the list. The program should meet the following requirements:

1. **Input:**
   - A list of integers, which can contain both positive and negative values.
   - An integer k, representing the position of the maximum value to find.
2. **Output:**
   - The k-th maximum value in the list.
   - If k is greater than the number of distinct elements in the list or if the list is empty, the program should return an appropriate message indicating the error.

Constraints:

- The list may contain duplicate values.
- The value of k should be a positive integer.
- The list may contain up to 10^6 elements, and each element can be as large as 10^9 in magnitude.

Examples:

Consider the following examples for better understanding:

1. **Example 1:**
   - **Input:** list = [3, 1, 5, 4, 2], k = 2
   - **Output:** 4
   - **Explanation:** The distinct elements in the list are [1, 2, 3, 4, 5]. The 2nd maximum value is 4.
2. **Example 2:**
   - **Input:** list = [7, 7, 7, 7, 7], k = 1
   - **Output:** 7
   - **Explanation:** The distinct elements in the list are [7]. The 1st maximum value is 7.
3. **Example 3:**
   - **Input:** list = [2, 1, 2, 1, 2], k = 3
   - **Output:** -1
   - **Explanation:** The distinct elements in the list are [1, 2]. There is no 3rd maximum value.

For example:

Input	Result
5 3 1 5 4 2 2	4
6 7 7 7 7 7 7 1	7
10 2 1 2 1 2 1 2 1 2 1 3	-1

Answer: (penalty regime: 0 %)

```
import sys

def find_kth_maximum(arr, k):
    # Get distinct elements using a set
    unique_elements = set(arr)

    # Check if k is valid relative to the number of distinct elements
    if k <= 0 or k > len(unique_elements):
        return -1

    # Sort distinct elements in descending order and return the k-th (1-indexed)
    sorted_unique = sorted(unique_elements, reverse=True)
    return sorted_unique[k - 1]

def main():
    # Read all tokens from standard input to handle competitive programming formats
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    iterator = iter(input_data)

    # Process inputs continuously
    while True:
        try:
            n_str = next(iterator)
        except StopIteration:
            break
```

```
        n = int(n_str)
        elements = [int(next(iterator)) for _ in range(n)]
        k = int(next(iterator))

        result = find_kth_maximum(elements, k)
        print(result)

if __name__ == "__main__":
    main()
```

	Input	Expected	Got	
✓	5 3 1 5 4 2 2	4	4	✓
✓	6 7 7 7 7 7 7 1	7	7	✓
✓	10 2 1 2 1 2 1 2 1 2 1 3	-1	-1	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **5**

Correct

Mark 1.00 out of 1.00

To find the frequency of numbers in a list and display in sorted order.

Constraints:

1<=n, arr[i]<=100

Input:

1 68 79 4 90 68 1 4 5

output:

1 2

4 2

5 1

68 2

79 1

90 1

For example:

Input	Result
4 3 5 3 4 5	3 2 4 2 5 2

Answer: (penalty regime: 0 %)

```
import sys
from collections import Counter

def main():
    # Read all tokens from standard input
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    # Count frequencies of all integers
    numbers = [int(x) for x in input_data]
    freq = Counter(numbers)

    # Sort keys in ascending order and print element along with its frequency
    for number in sorted(freq.keys()):
        print(f"{number} {freq[number]}")

if __name__ == "__main__":
    main()
```

	Input	Expected	Got	
✓	4 3 5 3 4 5	3 2 4 2 5 2	3 2 4 2 5 2	✓
✓	12 4 4 4 2 3 5	2 1 3 1 4 3 5 1 12 1	2 1 3 1 4 3 5 1 12 1	✓
✓	5 4 5 4 6 5 7 3	3 1 4 2 5 3 6 1 7 1	3 1 4 2 5 3 6 1 7 1	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **6**

Correct

Mark 1.00 out of 1.00

Given an array of integers `nums`, sort the array in **increasing** order based on the frequency of the values. If multiple values have the same frequency, sort them in **decreasing** order.

Print the *sorted array*.

Example 1:

```
Input:
6
1 1 2 2 2 3

Output:
3 1 1 2 2 2

Explanation: '3' has a frequency of 1, '1' has a frequency of 2, and '2' has a frequency of 3.
```

Example 2:

```
Input:
5
2 3 1 3 2
Output:
1 3 3 2 2
Explanation: '2' and '3' both have a frequency of 2, so they are sorted in decreasing order.
```

Example 3:

```
Input:
9
-1 1 -6 4 5 -6 1 4 1
Output:
5 -1 4 4 -6 -6 1 1 1
```

Constraints:

- `1 <= nums.length <= 100`
- `-100 <= nums[i] <= 100`

For example:

Input	Result
6 1 1 2 2 2 3	3 1 1 2 2 2
5 2 3 1 3 2	1 3 3 2 2

Answer: (penalty regime: 0 %)

```
import sys
from collections import Counter

def main():
    # Read all tokens from standard input
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    iterator = iter(input_data)

    # Process test cases continuously
    while True:
        try:
            n_str = next(iterator)
        except StopIteration:
            break

        n = int(n_str)
        nums = [int(next(iterator)) for _ in range(n)]

        # Count frequency of each number
        freq = Counter(nums)

        # Sort based on:
        # 1. Frequency (ascending) -> freq[x]
        # 2. Value (descending) -> -x
        nums.sort(key=lambda x: (freq[x], -x))

        # Print space-separated elements
        print(*nums)

if __name__ == "__main__":
    main()
```

	Input	Expected	Got	
✓	6 1 1 2 2 2 3	3 1 1 2 2 2	3 1 1 2 2 2	✓

	Input	Expected	Got	
✓	5 2 3 1 3 2	1 3 3 2 2	1 3 3 2 2	✓
✓	9 -1 1 -6 4 5 -6 1 4 1	5 -1 4 4 -6 -6 1 1 1	5 -1 4 4 -6 -6 1 1 1	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **7**

Correct

Mark 1.00 out of 1.00

Given an integer array `nums`, return *an integer array* `counts` *where* `counts[i]` *is the number of smaller elements to the right of* `nums[i]`.

Example 1:

```
Input: nums = [5,2,6,1]
Output: [2,1,1,0]
Explanation:
To the right of 5 there are 2 smaller elements (2 and 1).
To the right of 2 there is only 1 smaller element (1).
To the right of 6 there is 1 smaller element (1).
To the right of 1 there is 0 smaller element.
```

Example 2:

```
Input: nums = [-1]
Output: [0]
```

Example 3:

```
Input: nums = [-1,-1]
Output: [0,0]
```

Constraints:

- `1 <= nums.length <= 10^5`
- `-10^4 <= nums[i] <= 10^4`

For example:

Test	Result
`print(countSmaller([5,2,6,1]))`	`[2, 1, 1, 0]`
`print(countSmaller([-1]))`	`[0]`

Answer: (penalty regime: 0 %)

Reset answer

```python
import sys

def countSmaller(nums):
    n = len(nums)
    counts = [0] * n
    # Pair each element with its original index: (value, original_index)
    indexed_nums = [(nums[i], i) for i in range(n)]

    def merge_sort(arr):
        if len(arr) <= 1:
            return arr

        mid = len(arr) // 2
        left = merge_sort(arr[:mid])
        right = merge_sort(arr[mid:])

        merged = []
        i = j = 0

        # Merge step: track how many elements from right were smaller than left[i]
        while i < len(left) and j < len(right):
            if left[i][0] <= right[j][0]:
                counts[left[i][1]] += j
                merged.append(left[i])
                i += 1
            else:
                merged.append(right[j])
                j += 1

        while i < len(left):
            counts[left[i][1]] += j
            merged.append(left[i])
            i += 1

        while j < len(right):
            merged.append(right[j])
            j += 1

        return merged

    merge_sort(indexed_nums)
    return counts

def main():
    # Process standard input if available (for competitive programming judge)
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    iterator = iter(input_data)
    for token in iterator:
        # Check if the token represents an array size or single number
```

	Test	Expected	Got	
✓	`print(countSmaller([5,2,6,1]))`	`[2, 1, 1, 0]`	`[2, 1, 1, 0]`	✓
✓	`print(countSmaller([50,20,60,10]))`	`[2, 1, 1, 0]`	`[2, 1, 1, 0]`	✓
✓	`print(countSmaller([-1]))`	`[0]`	`[0]`	✓
✓	`print(countSmaller([-1, -1]))`	`[0, 0]`	`[0, 0]`	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **8**

Correct

Mark 1.00 out of 1.00

Bubble Sort is the simplest sorting algorithm that works by repeatedly swapping the adjacent elements if they are in wrong order. You read an list of numbers. You need to arrange the elements in ascending order and print the result. The sorting should be done using bubble sort.

Input Format: The first line reads the number of elements in the array. The second line reads the array elements one by one.

Output Format: The output should be a sorted list.

For example:

Input	Result
6 3 4 8 7 1 2	1 2 3 4 7 8
5 4 5 2 3 1	1 2 3 4 5

Answer: (penalty regime: 0 %)

```
import sys

def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                # Swap if adjacent elements are in wrong order
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr

def main():
    # Read all tokens from standard input
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    iterator = iter(input_data)

    # Continuously process test cases
    while True:
        try:
            n_str = next(iterator)
        except StopIteration:
            break

        n = int(n_str)
        elements = [int(next(iterator)) for _ in range(n)]

        sorted_elements = bubble_sort(elements)

        # Print space-separated sorted elements
        print(*sorted_elements)

if __name__ == "__main__":
    main()
```

	Input	Expected	Got	
✓	6 3 4 8 7 1 2	1 2 3 4 7 8	1 2 3 4 7 8	✓
✓	6 9 18 1 3 4 6	1 3 4 6 9 18	1 3 4 6 9 18	✓
✓	5 4 5 2 3 1	1 2 3 4 5	1 2 3 4 5	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **9**

Correct

Mark 1.00 out of 1.00

Given an integer array `nums` sorted in **non-decreasing** order, return *an array of **the squares of each number** sorted in non-decreasing order.*

Example 1:

```
Input: nums = [-4,-1,0,3,10]
Output: [0,1,9,16,100]
Explanation: After squaring, the array becomes [16,1,0,9,100].
After sorting, it becomes [0,1,9,16,100].
```

Example 2:

```
Input: nums = [-7,-3,2,3,11]
Output: [4,9,9,49,121]
```

Constraints:

- `1 <= nums.length <= 10^4`
- `-10^4 <= nums[i] <= 10^4`
- `nums` is sorted in **non-decreasing** order.

For example:

Test	Result
`print(sortedSquares([-4,-1,0,3,10]))`	`[0, 1, 9, 16, 100]`

Answer: (penalty regime: 0 %)

Reset answer

```
import sys

def sortedSquares(nums):
    n = len(nums)
    result = [0] * n
    left = 0
    right = n - 1

    # Fill the result array from right (largest square) to left (smallest square)
    for i in range(n - 1, -1, -1):
        left_square = nums[left] ** 2
        right_square = nums[right] ** 2

        if left_square > right_square:
            result[i] = left_square
            left += 1
        else:
            result[i] = right_square
            right -= 1

    return result

def main():
    # Process standard input if input tokens are present
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    iterator = iter(input_data)
    for token in iterator:
        try:
            n = int(token)
            nums = [int(next(iterator)) for _ in range(n)]
            print(sortedSquares(nums))
        except (StopIteration, ValueError):
            break

if __name__ == "__main__":
    main()
```

	Test	Expected	Got	
✓	`print(sortedSquares([-4,-1,0,3,10]))`	`[0, 1, 9, 16, 100]`	`[0, 1, 9, 16, 100]`	✓
✓	`print(sortedSquares([-7,-3,2,3,11]))`	`[4, 9, 9, 49, 121]`	`[4, 9, 9, 49, 121]`	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **10**

Correct

Mark 1.00 out of 1.00

Given an array of integers `arr`, replace each element with its rank.

The rank represents how large the element is. The rank has the following rules:

- Rank is an integer starting from 1.
- The larger the element, the larger the rank. If two elements are equal, their rank must be the same.
- Rank should be as small as possible.

Example 1:

```
Input: arr = [40,10,20,30]
Output: [4,1,2,3]
Explanation: 40 is the largest element. 10 is the smallest. 20 is the second smallest. 30 is the third smallest.
```

Example 2:

```
Input: arr = [100,100,100]
Output: [1,1,1]
Explanation: Same elements share the same rank.
```

Example 3:

```
Input: arr = [37,12,28,9,100,56,80,5,12]
Output: [5,3,4,2,8,6,7,1,3]
```

Constraints:

- `0 <= arr.length <= 10^5`
- `-10^9 <= arr[i] <= 10^9`

For example:

Test	Result
`print(arrayRankTransform([40,10,20,30]))`	`[4, 1, 2, 3]`

Answer: (penalty regime: 0 %)

Reset answer

```python
import sys

def arrayRankTransform(arr):
    # Map each unique element to its 1-based rank
    rank_map = {}
    rank = 1

    for val in sorted(set(arr)):
        rank_map[val] = rank
        rank += 1

    # Replace each original element with its corresponding rank
    return [rank_map[x] for x in arr]

def main():
    # Process standard input if input tokens are present
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    iterator = iter(input_data)
    for token in iterator:
        try:
            n = int(token)
            arr = [int(next(iterator)) for _ in range(n)]
            print(arrayRankTransform(arr))
        except (StopIteration, ValueError):
            break

if __name__ == "__main__":
    main()
```

	Test	Expected	Got	
✓	`print(arrayRankTransform([40,10,20,30]))`	`[4, 1, 2, 3]`	`[4, 1, 2, 3]`	✓
✓	`print(arrayRankTransform([100,100,100]))`	`[1, 1, 1]`	`[1, 1, 1]`	✓
✓	`print(arrayRankTransform([37,12,28,9,100,56,80,5,12]))`	`[5, 3, 4, 2, 8, 6, 7, 1, 3]`	`[5, 3, 4, 2, 8, 6, 7, 1, 3]`	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.